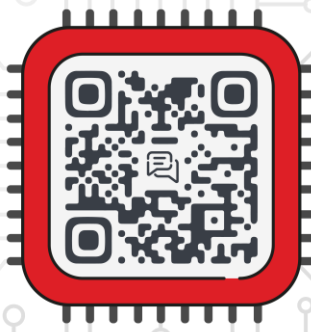




6

11

1320 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Multithreading

عملي مشترك



RB Informatics; 27/11/2024

برمجة ٣

...Multithreading ➤

“Thread”

← يستخدم thread class لجعل البرنامج قادر على تنفيذ عدة أوامر مع بعضها في وقت واحد، وهو يملك عدة دوال وخصائص تجعلنا قادرين على التحكم بطريقة تنفيذ هذه الأوامر.

🔗 سنقوم بمراجعة لـ Threads عن طريق مثال:

- ليكن لدينا ثلاثة توابع $m1()$ $m2()$ $m3()$ كل منها يأخذ وقت محدد للتنفيذ مثلاً $5s$ $2s$ $1s$ على الترتيب، فما هو الزمن الكلي اللازم لتنفيذ جميع هذه التوابع بالتتالي في حال استدعائها؟؟
 $1+2+5 = 8s$
- وفي هذه الحالة لم يكن لدينا سوى عامل واحد فقط في المعالج لتنفيذ جميع هذه التوابع.
- لكن في حال استطعنا توزيع هذه التوابع الثلاث على ثلاث عمال ألن يكون ذلك أفضل وأكثر كفاءة؟! كم سيكون الزمن الكلي لتنفيذ التوابع السابقة؟



العمال		الزمن اللازم لتنفيذ التابع: التوابع
W1	→	$m1() : 1s$
W2	→	$m2() : 2s$
W3	→	$m3() : 5s$

في هذه الحالة يكون الزمن الكلي هو 5 ثواني وذلك لأن العمال $W1$ $W2$ $W3$ يبدؤون بالعمل في نفس اللحظة وبالتالي الزمن الكلي لانتهاء العمل هو أطول زمن تنفيذ من بين الأزمنة الثلاث للتوابع السابقة وفي مثالنا هو $m3()$

ملاحظة!!
يمكننا القول أن كل عامل يمثل Thread
وكل تابع يمثل Task
أما العملية ككل فهي Process



خطوات إنشاء ال Thread و تشغيله:

1. ننشئ class أولاً وهذا ال class إما:
 ← الحالة '1': يرث من ال Thread class المعروف مسبقاً في لغة ال java .
 ← الحالة '2': ال class يقوم ب implements لل interface الذي يدعى Runnable.
 2. يجب أن نقوم ب @override للتابع public void run() ونضع بداخل هذا التابع التعليمات التي نريد تنفيذها.

الحالة 2

```
1 class MyThread implements Runnable{
2     @Override
3     public void run(){
4         // التعليمات التي نريد تنفيذها
5     }
6 }
7 public class Main{
8     public static void main (String [ ] args){
9         MyThread mth=new MyThread ( );
10        Thread th = new Thread(mth);
11        th.start ( );
12    }}
```

الحالة 1

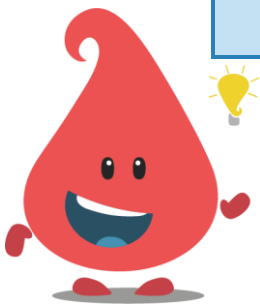
```
1 class MyThread extends Thread{
2     @Override
3     public void run(){
4         // التعليمات التي نريد تنفيذها
5     }
6 }
7 public class Main{
8     public static void main(string []args){
9         MyThread th = new MyThread ( );
10        th.start() ;
11    }
12 }
```

✓ ولكن يوجد طرق أخرى لإنشاء Thread أسهل وأبسط من الطرق السابقة:

الحالة '3': نعرف ال Thread مباشرة في تابع ال main ونكتب:

Thread th = new Thread (() -> { ... });

يتم وضع التعليمة المطلوبة
ضمن هذه الأقواس



الحالة '4': نعرف ال Thread في تابع ال main ونكتب:

```
Thread th = new Thread ( __ :: __ );
```

اسم ال class

اسم التابع المراد استدعاه
ضمن الكلاس

الحالة 3

```
1 class MyThread {
2
3     public static void run(){
4         // التعليمات التي نريد تنفيذها
5     }
6 }
7 public class Main{
8     public static void main(String []args){
9         Thread mth = new Thread ( ( ) -> { MyThread.run();} );
10        mth.start();
11    } }
```

الحالة 4

```
1 class MyThread {
2
3     public static void run(){
4         // التعليمات التي نريد تنفيذها
5     }
6 }
7 public class Main{
8     public static void main (String [ ] args){
9         Thread mth=new Thread ( Mythread :: run );
10        mth.start ( );
11    } }
```



بالطبع ستتعب..

لكن لو كان النجاح سهلا لوصل إليه الجميع!

وليس الجميع مثلك ♥

■ التوابع الخاصة بـ Thread :

public void start()

تستخدم لتشغيل كائن الـ Thread أي تنفيذ الأوامر التي في الدالة `run()` و ترمي استثناء من نوع:

`IllegalThreadStateException`

في حال كان الـ Thread شغالاً قبل استدعائها.

public void run()

يتم تنفيذ التعليمات داخل التابع `run()` فور تشغيل الـ Thread

public void sleep(long milli seconds)

توقف تنفيذ تعليمات الـ Thread التي قامت باستدعائها لمدة محددة ثم يعود للتنفيذ بعد انتهاء المدة

public void stop()

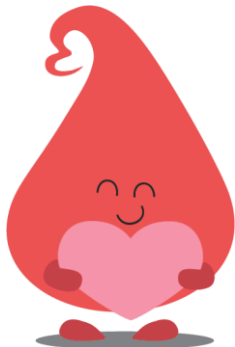
توقف عمل الـ Thread التي قامت باستدعائها بشكل نهائي

public void join()

توقف البرنامج من تنفيذ التعليمات الموجودة ضمن تابع الـ `main`، (`mainthread`) حتى تنتهي جميع التعليمات الموجودة في هذه الـ Thread.

In a world where you can be anything.

Be Good



تمارين.. ➤

1. اكتب الصف Example1 الذي يمكن تنفيذه كمهمة مستقلة Thread ويقوم بطباعة جملة " Hello from "mythread".

```
1 package main1;
2 public class Example1 implements
   Runnable {
3     public void run() {
4         System.out.println("Hello from my thread.");
5     }
6     public static void main(String args[]) {
7         Example1 obj = new Example1();
8         Thread tobj = new Thread(obj);
9         tobj.start();
10    }
11 }
```

السطر (2): تعريف الصف Example1 والذي يحقق الواجهة Runnable لكي تستطيع تنفيذه ك Thread

السطر (3,4): تعريف التابع run() الذي يقوم بطباعة العبارة المطلوب { نضع ضمنه التعليمات التي نريد تنفيذها داخل ال thread }

السطر (6 الى 10): تعريف التابع الرئيسي الذي يقوم ببناء كائن من الصف Example1 وبناء Thread وتشغيله { عند استدعاء التابع start() يبدأ تنفيذ ال Thread }.

You have to be ODD to be No.1



2. اكتب الصف Example2 الذي يمكن تنفيذه كمهمة مستقلة Thread ويحوي خاصية ويقوم بطباعة اسم المهمة مع رقم من 1 إلى 4، لكن بين كل عملية طباعة يوجد غفوة لمدة ثانية. ثم قم بإنشاء الصف الرئيسي main ويقوم بإنشاء مهمتين من الصف Example2 وتشغيلهما في نفس الوقت.

```

1 package main2;
2 class Example2 implements Runnable {
3     private Thread t;
4     private String threadName;
5     Example2(String name) {
6         threadName = name;
7         System.out.println("Creating " + threadName);
8     }
9     public void run() {
10        System.out.println("Running " + threadName);
11        try {
12            for (int i = 4; i > 0; i--) {
13                System.out.println("Thread: " +
14                    threadName + ", " + i);
15                Thread.sleep(1000);
16            } catch (InterruptedException e) {
17                System.out.println("Thread " +
18                    threadName + " interrupted.");
19            }
20            System.out.println("Thread " +
21                threadName + " exiting.");
22        }
23        public void start() {
24            System.out.println("Starting " + threadName);
25            if (t == null) {
26                t = new Thread(this, threadName);
27                t.start();
28            }
29        }
30        public class Main2 {
31            public static void main(String[] args) {
32                Example2 T1 = new Example2("Thread-1");
33                T1.start();
34                Example2 T2 = new Example2("Thread-2");
35                T2.start();
36            }
37        }
38    }
39 }

```

السطر (2): تعريف الصف Example2 والذي يحقق

الواجهة Runnable لكي تستطيع تنفيذه ك Thread

السطر (3): تعريف Thread ضمن الصف Example2.

السطر (4): تعريف خاصية اسم المهمة thread

name

السطر (5)---(7): تعريف الباني الذي يهيئ اسم

المهمة ويطبّع عبارة تدل على إنشاء مهمة.

السطر (9)---(10): تعريف التابع run() وطباعة رسالة

تدل على بدء التنفيذ.

السطر (11)---(18): طباعة اسم المهمة مع رقم من 4

حتى 1 مع غفوة بين كل عملية طباعة وأخرى بالإضافة

إلى معالجة الاستثناء المتوقع.

السطر (19): طباعة عبارة تدل على انتهاء المهمة.

السطر (21)---(26): تعريف التابع start الذي يهيئ

المهمة ويقوم بتشغيلها في حال كانت غير مهيئة من

قبل.

السطر (27)---(33): تعريف الصف الرئيسي مع التابع

الرئيسي الذي يقوم بإنشاء مهمتين من الصف

Example2 باسمين مختلفين وتشغيلهما.

3. اكتب الصف Count الذي يمكن تنفيذه بواسطة الباني الخاص به كمهمة مستقلة Thread ويقوم بطباعة الأرقام من ال 0 إلى 9 ولكن بين كل عملية طباعة يوجد غفوة لمدة ثانية. ثم قم بإنشاء الصف الرئيسي main الذي يحوي التابع الرئيسي main ويقوم بإنشاء مهمة من الصف Count و طباعة عبارة كل ثانية ونصف طالما أن المهمة Count لم تنته.

```

1 package main3;
2 class Count implements Runnable {
3     Thread mythread;
4     Count() {
5         mythread = new Thread(this, "my runnable
6         thread");
7         System.out.println("my thread created" +
8         mythread);
9         mythread.start();
10    }
11    public void run() {
12        try {
13            for (int i = 0; i < 10; i++) {
14                System.out.println("Printing the count " + i);
15                Thread.sleep(1000);
16            }
17        } catch (InterruptedException e) {
18            System.out.println("my thread interrupted");
19        }
20        System.out.println("mythread run is over");
21    }}
22 class Main {
23    public static void main(String args[]) {
24        Count cnt = new Count();
25        try {
26            while (cnt.mythread.isAlive()) {
27                System.out.println("Main thread will
28                be alive till the child thread is live");
29                Thread.sleep(1500);
30            }
31        } catch (InterruptedException e) {
32            System.out.println("Main thread interrupted");
33        }
34        System.out.println("Main thread run is over");
35    }}

```

السطر (2): تعريف الصف Count والذي يحقق

الواجهة Runnable لكي تستطيع تنفيذ ال Thread .

السطر (3): تعريف Thread ضمن الصف count .

السطر (4)---(7): تعريف الباني الذي يهيئ اسم

المهمة ويقوم بتشغيلها.

السطر (9)---(17): تعريف التابع run الذي يقوم

بطباعة الأرقام من 0 حتى 9 مع غفوة لمدة ثانية بين كل عملية طباعة و معالجة الاستثناء المتوقع.

السطر (18): طباعة عبارة تدل على انتهاء المهمة.

السطر (20)---(32): تعريف الصف الرئيسي مع

التابع الرئيسي الذي يقوم بإنشاء مهمة من الصف Count ويقوم بطباعة عبارة ثابتة كل ثانية و نصف طالما لم تنته هذه المهمة من عملها و معالجة الاستثناء المتوقع.

synchronization

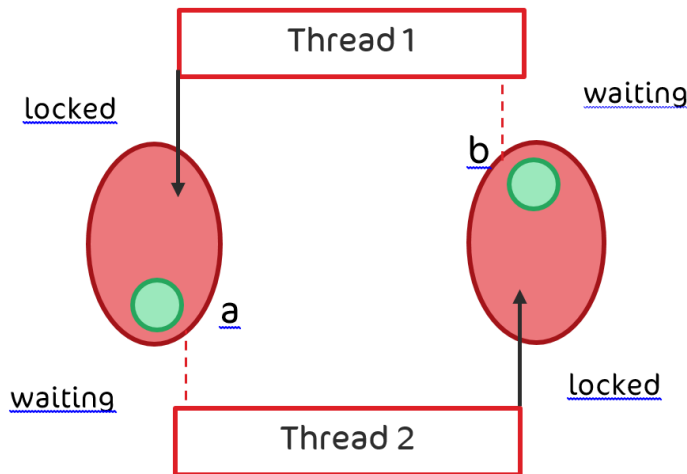
أحياناً قد نحتاج أكثر من Thread لنفس المصدر وبالتالي قد يؤدي لحدوث مشاكل، وهنا أتت أهمية الـ **synchronization** وعمله كالتالي:

حيث عند وصول أحد الـ Thread إلى ذلك المصدر يحدث " قفل " عليه بالنسبة لباقي الـ thread، أي عند استخدام **synchronization keyword** ستنفذ Thread واحد في كل مرة (ينفذ الـ Thread الأول بشكل كامل ثم ينفذ الآخر).

Dead lock ✓

لدينا Resource a , Resource b و thread1 , thread2 :

حيث أن thread1 ينفذ lock على Resource a و thread2 ينفذ lock على Resource b ولكن كل من two threads يحتاج لمصدر آخر وفي هذه الحالة تحدث **Dead lock** أي كليهما ينتظر الآخر



Find light in the beautiful sea

I choose to be happy

You and I, you and I, we're like **DIAMONDS** in the sky

4. اكتب الصف Threadsend كمهمة مستقلة Thread ويحوي الوصفة رسالة msg ويقوم بإرسال هذه الرسالة بحيث يقوم بطباعة عبارة تدل على أنه جاري إرسال ال msg ثم ينام لمدة ثانية واحدة وبعدها يقوم بطباعة عبارة تدل على أنه تم إرسال ال msg ولكن يجب منع تنفيذ مهمة أخرى أثناء عملية الإرسال السابقة من خلال استخدام Synchronized ثم قم بكتابة الصف الرئيسي الذي يقوم بإرسال رسالتين في نفس الوقت.

```

1 package main4;
2 class Sender {
3     synchronized void send(String msg) {
4         System.out.println("Sending\t" + msg);
5         try {
6             Thread.sleep(1000);
7         } catch (Exception e) {
8             System.out.println("Thread interrupted.");
9         }
10        System.out.println("\n" + msg + "Sent");
11    }
12 class ThreadedSend extends Thread {
13     private String msg;
14     Sender sender;
15     ThreadedSend(String m, Sender obj) {
16         msg = m;
17         sender = obj;
18     }
19     public void run() {
20         sender.send(msg);
21 public class Main {
22     public static void main(String args[]) {
23         Sender snd = new Sender();
24         ThreadedSend S1 = new ThreadedSend(" Hi
25         ThreadedSend S2 = new ThreadedSend(" Bye
26         S1.start();
27         S2.start();
28         try {
29             S1.join();
30             S2.join();
31         } catch (Exception e) {
32             System.out.println("Interrupted");

```

السطر (2): تعريف الصف Sender والذي يحوي

تابع الإرسال المتزامن synchronized أي أن هذا التابع لا يمكن أن ينفذ مهمتين في نفس الوقت وبالتالي يتنفذ كتلة واحدة حتى مع وجود غفوة بداخله.

السطر (3-11): تعريف التابع send الذي يقوم

بطباعة رسالة البدء ثم يغفو لمدة ثانية واحدة قبل طباعة رسالة الانتهاء من الإرسال.

السطر (12): تعريف الصف ThreadedSend

والذي يرث من الصف Thread لكي تستطيع تنفيذه كمهمة مستقلة Thread.

السطر (13-18): تعريف واصفة الرسالة وكانت

من نمط sender بالإضافة للباني الذي يهيئ هذه الواصفات.

السطر (19 ، 20): تعريف التابع (run) الذي يقوم

باستدعاء تابع الإرسال.

السطر (21-32): تعريف الصف الرئيسي مع

التابع الرئيسي الذي يقوم بإنشاء مهمتين من الصف ThreadSend برسالتين مختلفتين وتشغيلهما ثم الانتظار حتى انتهاء المهمتين مع معالجة الاستثناء المتوقع.

5. اكتب برنامج يقوم بإنشاء مهمتين الأولى تدور في حلقة عشر مرات ومن أجل كل دورة نقوم بطباعة عدد صحيح من 0 ل99 ثم تنام لمدة ثانية واحدة، أما المهمة الثانية فتقوم بنفس العمل ولكن مع طباعة عدد حقيقي عشوائي بدلاً من عدد صحيح.

File '1': class print

```

1 package main;
2 import java.util.Random;
3 public class Print implements Runnable {
4     @Override
5     public void run() {
6         try {
7             Random r1 = new Random();
8             int d = 0;
9             for (int j = 0; j < 10; j++) {
10                 d = r1.nextInt(0,99);
11                 System.out.println(d);
12                 Thread.sleep(1000);
13             }
14         } catch (InterruptedException e) {
15             System.out.println("interrupted thread");
16         }
17     }
18 }

```

File '2': class print1

```

1 package main;
2 import java.util.Random;
3 import java.util.logging.Level;
4 import java.util.logging.Logger;
5
6 public class Print1 implements Runnable {
7     @Override
8     public void run() {
9         try {
10             Random r = new Random();
11             double k;
12             for (int i = 0; i < 10; i++) {
13                 k = r.nextDouble(0,99);
14                 System.out.println(k);
15                 Thread.sleep(1000);
16             }
17         } catch (InterruptedException ex) {
18             System.out.println("Interrupted thread");
19         }
20     }
21 }

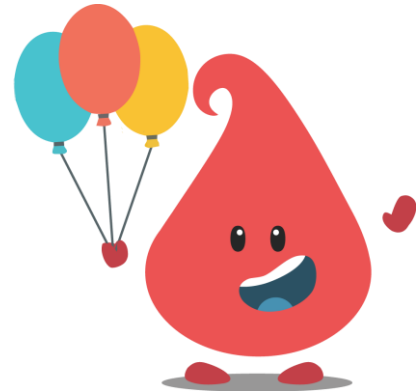
```

File '3': class Main

```

1 package main;
2 import java.util.Random;
3
4 public class Main {
5     public static void main(String[] args) {
6         Print p = new Print();
7         Thread t = new Thread(p);
8         t.start();
9         Print1 p1 = new Print1();
10        Thread t1 = new Thread(p1);
11        t1.start();
12    }
13 }

```



أولاً يدخل ال compiler إلى تابع ال main ثم بالسطر (6) من file(3) عرفنا object لل Print class ثم مررناه ك parameter لل constructor الخاص ب thread كما في السطر (7)

ثم تم استدعاء التابع start() الذي يقوم بتشغيل ال thread ويدخل ال file(1) وسنقوم بتشغيل تابع run() بالسطر (5) الذي يوضع بداخله التعليمات التي نريدها

ثم بالسطر (7) عرفنا obj من كلاس Random وتعريف متحول وحلقة تقوم بطباعة قيمة عشوائية من نوع int ضمن المجال (0<---99) ثم ننام لمدة ثانية وذلك من السطر (8<---12)

ثم بالسطر (13): معالجة Exception من نوع interrupted Exception المتوقع ظهوره.

ثم يعود لتابع ال main بالسطر (9) من ال file(3) تقوم بإنشاء obj من كلاس Print2 وتمرره أيضاً لكلاس Thread ونقوم باستدعاء تابع ال start() ويدخل لل file(2) ثم للتابع run() بالسطر (7) التي تحوي try block و عرفنا ضمن try (object) من كلاس Random وعرفنا متحول k بالسطر (10)

السطر (11<---14): حلقة for تقوم بطباعة قيمة عشوائية من نوع double ثم تنام لمدة ثانية واحدة.

السطر (15): معالجة Exception من نوع interrupted Exception المتوقع ظهوره وينهي البرنامج.



ملاحظة!

يمكن استخدام تعليمة:

Thread.activeCount();

والتي تعيد عدد ال Threads التي تعمل في البرنامج.



See You Soon..

